桂林電子科技大学
GUILIN UNIVERSITY OF ELECTRONIC TECHNOLOGY

《微机原理》课程大作业
报告

题目：基于 ESP32 CSI 扰动特征的多监测点监控系统

学　　号：【实测待填：学号】

姓　　名：【实测待填：姓名】

小 组 成 员：【实测待填：小组成员】

学　　院：【实测待填：学院】

专　　业：【实测待填：专业】

指 导 教 师：【实测待填：指导教师】

【实测待填：日期】

摘　要

本课题为《微机原理》课程大作业题目 3：设计并实现一套基于 Wi-Fi 信道状态信息（CSI）扰动特征的多监测点监控系统。系统由 3 个发送节点（2 块经典 ESP32、1 块 ESP32-C3）与 1 个 ESP32-S3 接收节点组成：发送节点以约 100 Hz 持续发包，接收节点用 esp-radar 组件按源 MAC 将 CSI 分发到各监测点链路，逐链路完成滑动窗均值/方差/峰峰值统计与滞回状态机判决，经 2 Mbps UART 二进制协议（帧同步 AA 55、CRC-16/CCITT-FALSE 校验、5 类帧）上报 PC 上位机（PySide6+pyqtgraph），实现区域状态卡片、实时特征曲线、事件日志与 CSV 记录/回放；标定阈值与链路表经 NVS 断电保存。台架实测：协议自测 20/20 通过；单链路 60 s 连续采集 546 帧无断流、包间隔中位数 110 ms；60 s 心跳 59 帧、特征帧 533 帧、零 CRC 错；双链路标定阈值 681/20367（centi-var）并经 TX 复位静默窗验证事件 MAC 归因无误；断电重启后链路表与阈值 1.26 s 内从 NVS 恢复；上位机冒烟测试 17/17 通过。系统功能链路已在台架全部打通，空间部署后的现场指标验收待执行。

关键词：WiFi CSI；ESP32；多节点；扰动检测；UART

目　录

1　引言 1
1.1 背景 1
1.2 选题与任务 1
1.3 本文工作与组织 1
2　设计指标 3
2.1 题目要求逐条落实与指标承诺 3
3　硬件平台与开发环境 4
3.1 硬件平台 4
3.2 开发环境 4
3.3 系统连接 4
4　系统设计 6
4.1 顶层架构 6
4.2 二进制上报协议 6
4.2.1 帧格式与帧类型 6
4.2.2 串口流式解析与失步重同步 8
4.3 扰动检测算法 8
4.3.1 滑动窗统计与 RSSI 突变过滤 8
4.3.2 滞回状态机 9
4.4 现场标定与持久化 9
4.5 系统设计与微机原理知识点的对应 10
4.6 上位机设计 10
5　关键模块实现 11
5.1 固件（ESP32_CIS_V1.0） 11
5.1.1 cis_detect：累加器与滑窗统计 11
5.1.2 cis_report：帧发送与心跳 13
5.1.3 cis_console：双 REPL 14
5.1.4 cis_peer：链路表与 NVS 持久化 14
5.1.5 cis_wifi：连网与兜底 15
5.2 上位机（pc_dashboard） 15

5.2.1 protocol：协议编解码与流式解析 15
5.2.2 serial_source：双线程与环形缓冲 16
5.2.3 store：CSV 会话与回放缓冲 16
5.2.4 ui_main：监控界面 16
6 测试与验证 18
6.1 协议自测 18
6.2 数据流与帧质量 18
6.3 双链路联调与 NVS 恢复 18
6.4 上位机冒烟 19
6.5 台架验证小结 19
7 现场部署与验收 20
7.1 部署点位 20
7.2 现场标定结果 20
7.3 七条验收用例结果 20
7.4 30 分钟长稳与上位机实机运行 21
8 总结与展望 22
8.1 工作总结 22
8.2 局限与已知问题 22
8.3 展望 22
9 分工 24
附录 A 协议帧格式总表 25
附录 B 工程目录与证据文件索引 26
B.1 工程目录索引 26
B.2 证据文件索引 26
B.3 【实测待填】占位清单 27
参考文献 28

1 引言

1.1 背景

Wi-Fi 信道状态信息（Channel State Information, CSI）刻画了无线信号从发送端到接收端经历的多径叠加结果：环境中人或物体的移动会改变多径传播，进而引起 CSI 幅值与相位的波动。利用这种“无需被测设备携带任何传感器”的无线感知（Wi-Fi Sensing）能力，可以用普通的商用 Wi-Fi 芯片实现运动检测、存在检测等应用[2]。IEEE 于 2025 年 5 月正式批准了 802.11bf WLAN Sensing 标准[3]，标志着该方向已从学术研究进入行业标准化阶段。

在工程侧，乐鑫（Espressif）官方开源工程 esp-csi 为 ESP32 系列芯片提供了 CSI 采集与检测组件（esp-radar）[1]，使廉价微控制器可以承担边缘感知任务[2]。社区实践（如 ESPectre 运动检测框架[4]）与学术研究同时指出两个关键约束：其一，ESP32 的 CSI 不携带 AGC 增益信息，射频前端增益切换会造成乘性幅值突变，需在检测端加以过滤[7,8]；其二，商品级 ESP32 在多人/多目标分辨上是公认弱项，多节点系统应以“每条链路独立检测”管理预期[5]。

1.2 选题与任务

本组选题为课程大作业题目 3：**基于 ESP32 CSI 扰动特征的多监测点监控系统**。题目要求概述如下：发送节点分布于各监测区，一块 ESP32-S3 接收节点利用 esp-radar 组件的多 peer 能力按源 MAC 分发 CSI，对每条链路独立完成运动/存在检测，并经 UART 二进制协议上报；PC 端 Python 桌面监控程序展示区域状态卡片、实时特征曲线与事件日志，支持 CSV 记录与回放。

本组对该题目的理解为：这是一个“多节点无线感知 + 嵌入式实时检测 + 串行通信 + 上位机可视化”的综合系统设计题，核心难点在于（1）多发送节点共享同一信道时，接收端必须能按来源区分链路并互不串扰；（2）检测算法须在廉价微控制器上实时运行，且对环境变化可现场标定；（3）上行链路须在控制台日志与二进制帧交织的真实串口流上保证数据完整性；（4）作为微机原理课程大作业，系统须覆盖串行通信、中断与回调、定时器、存储器组织、多任务与并发保护等课程知识点。

1.3 本文工作与组织

本文完成了题目 3 系统的设计、实现与台架联调。主要工作包括：搭建“1 收 3 发 + 路由器组网 + 直收兜底”的系统拓扑；设计带 CRC 校验与失步重同步的 UART 二进制上报协议；实现逐链路滑动窗统计与滞回状态机检测算法（含 RSSI 突变帧过滤）；实现现场标定与 NVS 持久化；开发 PySide6 桌面上位机；完成协议自测、数据流、帧质

量、双链路联调与上位机冒烟共五组台架实验。

本文组织如下：第 2 章给出设计指标；第 3 章介绍硬件平台与开发环境；第 4 章阐述系统设计（含微机原理知识点映射）；第 5 章介绍关键模块实现；第 6 章汇总已完成的台架测试与验证；第 7 章为现场部署与验收安排（待执行）；第 8 章总结全文并讨论局限；第 9 章为小组分工；附录 A/B 给出协议帧格式总表与工程文件索引。

2 设计指标

2.1 题目要求逐条落实与指标承诺

依据题目 3 要求与本组理解，系统须同时满足“多监测点区分”“实时检测上报”“可靠传输”“现场可标定”“断电可恢复”五类能力。表2.1给出逐条指标承诺、对应的实现机制与验证落点。其中带 † 的指标已在台架联调中验证（第6章），带 ∗ 的指标依赖空间部署后的现场验收（第7章，待执行）。

表 2.1 设计指标承诺表

指标项	承诺值 / 实现机制	验证落点
监测点数	3 条链路（TX1/TX2/TX3），固件按源 MAC 分发，容量 8 槽	§4.1、§6.3†
特征流速率	每 peer 10 Hz（1–50 可调），检测窗约 10 包/窗	§4.3†
运动事件响应	≤ 2 s（N=10 窗确认 ≈1 s + 2 Hz 重发）	§4.3∗
停走回归事件	≤ 3 s（M=25 窗 @10 Hz = 2.5 s）	§4.3∗
丢帧率	10 min 内 < 1%（三层丢帧计数）	§6.2†、§7.3∗
断电恢复	≤ 30 s（NVS 持久化，实测 4.9 s 联网、1.26 s 链路表恢复）	§6.3†
现场标定	每链路 ≤ 30 s（train 15 s + 保存），挪动即作废重标	§4.4
验收方式	台架（功能/数据流）与场地（七条用例）两级验收	§7.3∗

需要说明的是：运动/存在类指标的最终数值依赖真实空间布局下的噪声底，近距台架（板间距 < 0.5 m、RSSI 饱和）只能验证功能正确性，不能给出灵敏度结论；相关验收按第7章的七条用例在现场执行。

3 硬件平台与开发环境

3.1 硬件平台

系统共使用 4 块开发板：1 块 ESP32-S3 作为接收节点（RX），2 块经典 ESP32 与 1 块 ESP32-C3 作为发送节点（TX1–TX3），另有一块 ESP32-S3 作热备。表3.1给出板卡清单。板卡均为裸模组形态，除 Flash/RAM 外不假设任何板载外设；各板的物理 MAC 以 esptool 实测读出为唯一身份锚点（COM 口号会随重插变化，不可作为身份依据）。

表 3.1 板卡清单与身份映射

板	角色	芯片	物理 MAC（esptool 实测）	CSI 源 MAC
S3#1（16 MB Flash）	RX	双核 Xtensa LX7	DC:DA:0C:1D:AE:10	—
经典 #1	TX1	ESP32	8C:94:DF:A1:DD:D0	1a:00:00:00:00:00
C3	TX2	单核 RISC-V	E4:B0:63:33:F0:40	1a:00:00:00:00:01
经典 #2	TX3	ESP32	8C:94:DF:A1:DA:4C	烧前须改为 1a:00:00:00:00:02

三点说明：（1）**CSI 源 MAC 与物理 MAC 不同**——TX 固件内经 `esp_wifi_set_mac` 有意设置了本地管理地址作为 CSI 帧的源 MAC，RX 端注册 peer、按源 MAC 分发、事件/特征帧携带的 MAC 均为该 CSI 源 MAC；各 TX 源 MAC 必须互不相同，否则多链路在 RX 端混叠。（2）TX3（经典 #2）联调期间过热进不了下载模式，待冷却重插后先烧轻载固件再转正式任务，三链路完整联调待其恢复。（3）RX 的 S3 模组配 16 MB Flash 与 8 MB Octal PSRAM，固件使用自定义分区表（factory@0x20000、nvs、coredump），IPEX 外置天线接口保证射频裕量；BOOT/EN 按键用于下载模式与复位。

3.2 开发环境

3.3 系统连接

系统拓扑如图3.1所示。全部节点以 STA 模式关联路由器 1003-k，从而工作在同一信道（跟随 AP，实测信道 4；部署时按标定 SOP 将路由器手动固定信道与 20 MHz 带宽）；CSI 感知链路本身为 TX→RX 的同信道直收，不依赖路由器转发。RX 另开 SoftAP “CIS_RX-ae10” 作为无路由场地的兜底接入；预留直收模式（固定信道混杂接收，编译期选择）应对答辩场地无路由器的情况。上行通道以 COM3 串口（USB-Serial-JTAG，2,000,000-8-N-1）为主，UDP 3333 为可选副通道，TCP 2323 提供无线 REPL 控制台。

实物连接照片（RX/TX 板卡、天线、整机）见图3.2。

表 3.2 开发环境与工具链

项	配置
固件框架	ESP-IDF v5.5.6（确定性环境脚本激活）
CSI 组件	esp-radar 0.3.4（esp-csi 官方组件，组件管理器缓存）
RX 工程	`ESP32_CIS_V1.0`（目标 esp32s3，自研模块 cis_*）
TX 工程	`csi_tx_v1`（经典 ESP32）、`csi_tx_c3_v1`（ESP32-C3），均为官方 csi_send 示例改配置
上位机	Python：PySide6 + pyqtgraph + pyserial（协议编解码纯标准库）
烧录与备份	`scripts/flash_rx.bat`、`scripts/flash_tx.bat`（内置 MAC 门禁与源 MAC 防呆）、`scripts/backup_flash.sh`（烧录前整片备份）
组网	路由器 1003-k（2.4 GHz，20 MHz；STA 信道自动跟随 AP，实测信道 4）

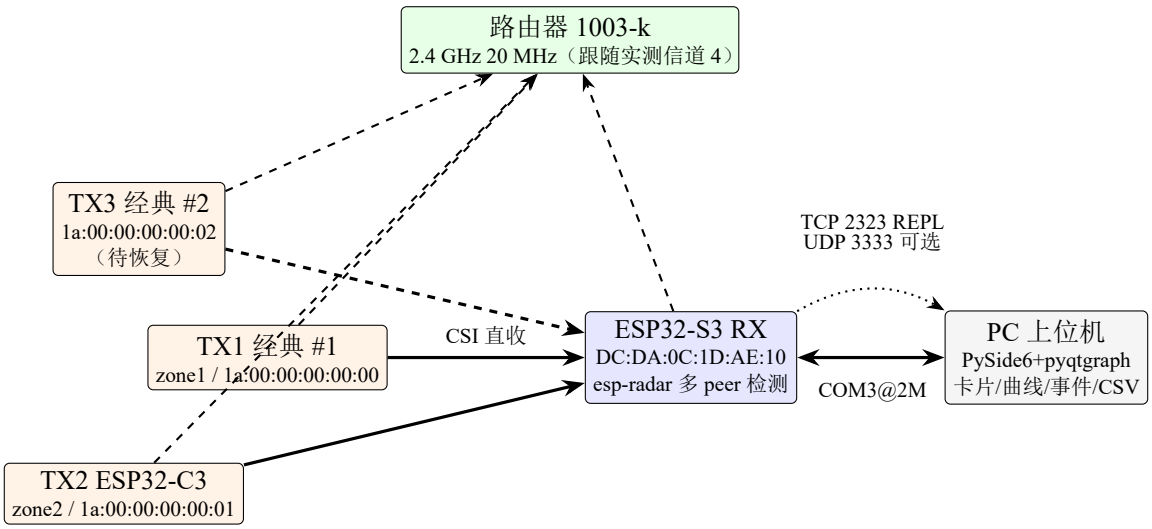


图 3.1 系统拓扑：1 收 3 发 + 路由器组网 + 串口主上行（虚线为各节点 STA 关联路由器、信道跟随 AP；CSI 链路为 TX→RX 同信道直收，不依赖路由器转发）

【实测待填：实物照片——RX（ESP32-S3+ 外置天线）、TX 板卡与整机连接】

图 3.2 系统实物连接照片（待现场拍摄替换）

4 系统设计

4.1 顶层架构

系统采用“1 收 N 发、按源 MAC 分发”的架构，如图4.1所示。设计要点有四：

（1）**链路区分**。各 TX 固件以互不相同的本地管理地址（CSI 源 MAC）发包；RX 端 esp-radar 组件以 `filter_mac` 做引导过滤，注册精确 peer 后由其按源 MAC 分发，每个源 MAC 对应一条独立检测链路。不使用通配 peer：实测若把 FF:FF:FF:FF:FF:FF 设为默认 peer，同信道全部邻居流量（一屏 7+ 个 MAC）都会进入处理管线，检测窗被“Timestamp delta out of range” 持续清空、回调饿死。故默认 peer 固定为 TX1 源 MAC 作引导，其余链路经 REPL 命令注册。

（2）**检测自管**。检测统计、状态机、阈值与训练全部由自研模块 cis_detect 管理（per-peer 上下文），不触碰 esp-radar 自带的单例检测器，避免多 peer 互相覆盖。

（3）**组网双模**。主模式为路由器模式：全节点连 1003-k，STA 信道自动跟随 AP，无需硬编码；备份模式为直收模式（编译期 `CONFIG_CIS_MODE_DIRECT`），TX 定信道广播、RX `esp_wifi_set_channel` 固定信道混杂直收，应对无路由器场地。RX 另开 SoftAP 兜底。

（4）**通道分层**。上行以串口为主通道（2 Mbps，带宽余量充足）；TCP 2323 REPL 与 UDP 3333 仅同网段可达（实测 PC 与节点网段隔离），故无线通道定位为“标定/调试”而非数据主通道。

4.2 二进制上报协议

4.2.1 帧格式与帧类型

上行帧格式如图4.2所示：2 字节同步字 AA 55，1 字节协议版本（0x01），1 字节帧类型，2 字节小端载荷长度，随后为载荷，帧尾为 2 字节小端 CRC16。CRC 采用 CRC-16/CCITT-FALSE（poly 0x1021、init 0xFFFF、不反射、无终值异或），覆盖范围从 ver 起至载荷末字节（不含同步字与 CRC 本身），标准校验值 `crc16(b"123456789") == 0x29B1`；固件与上位机两端以同一实现对照（`main/cis_protocol.h` ↔ `pc_dashboard/protocol.py`，2026-10-28 逐字段比对一致）。

帧类型共 5 类，见表4.1。数值统一采用 centi 刻度定点化：mean/p2p 为 centi-amp（raw×100），var/score/阈值为 centi-var（raw^2×100），超出 u16 范围饱和为 0xFFFF。定点化使帧长固定（特征帧仅 17 B），10 Hz/peer 下的上行带宽约 1 KB/s，2 Mbps 串口余量充足。

其中事件帧 state 字段取 0/1/2 分别对应静止（IDLE）、运动（MOTION）、存在（PRESENCE），score 为触发时窗方差（centi-var），与阈值直接可比。固件侧 CRC 实现

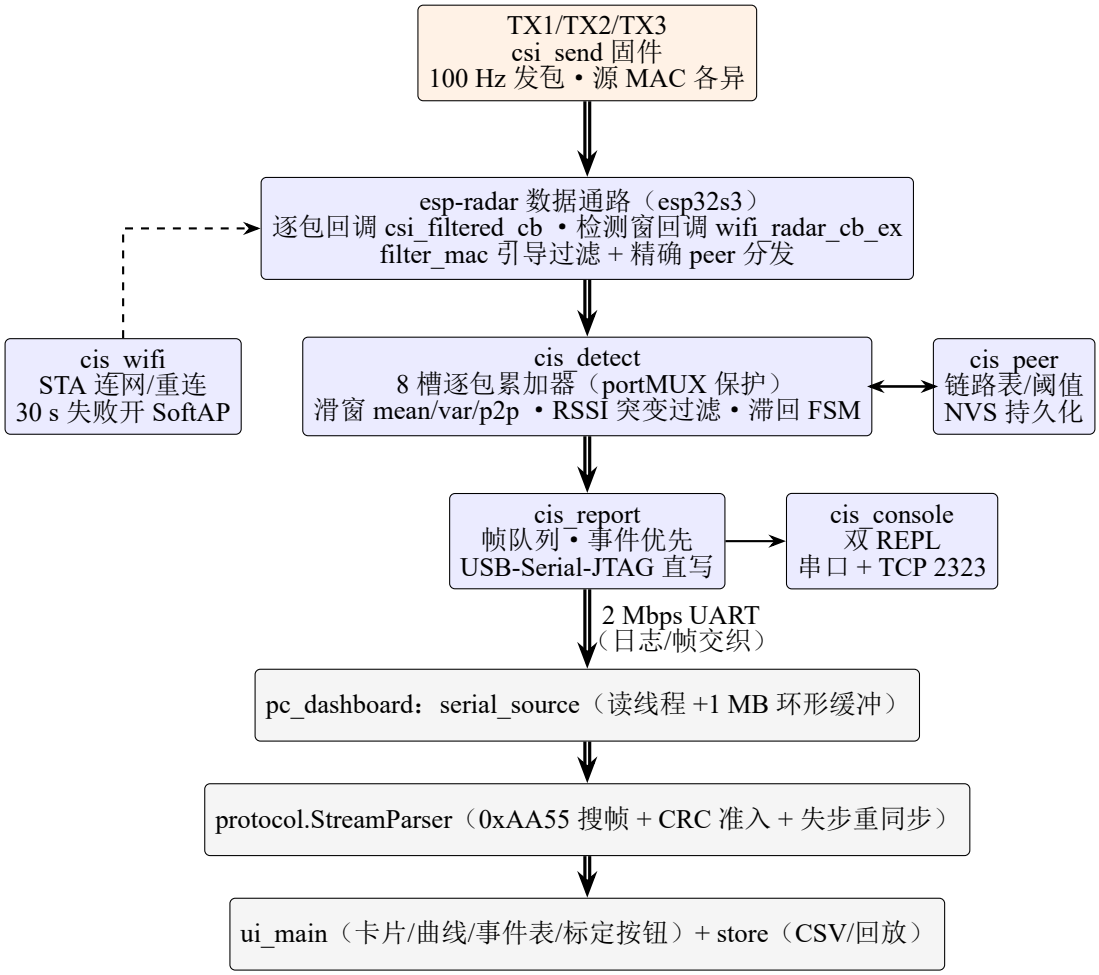


图 4.1 系统顶层架构与数据流



图 4.2 上行二进制帧格式

表 4.1 帧类型与载荷定义（多字节字段一律小端）

类型	名称	载荷字段	频率
0x01	心跳	uptime_ms(4) peer_count(1) free_heap(4) lost_frames(2)，11 B	1 Hz
0x10	事件	mac(6) name_id(1) state(1) score(2) rssi(1) ts_ms(4)，15 B	跳变即时 +MOTION 2 Hz
0x11	特征流	mac(6) ts_ms(4) mean(2) var(2) p2p(2) rssi(1)，17 B	10 Hz/peer
0x20	日志	ASCII 文本（变长，train 提示等）	按需
0x30	标定	mac(6) train_s(2) threshold(2) noise_floor(2)，12 B	train 结束

如下（cis_protocol.h）：

代码 4.1 CRC-16/CCITT-FALSE 固件实现（cis_protocol.h）

```
/* CRC16-CCITT-FALSE: poly 0x1021, init 0xFFFF */
static inline uint16_t cis_crc16_ccitt(const uint8_t *data, size_t
    len)
{
    uint16_t crc = 0xFFFF;
    for (size_t i = 0; i < len; i++) {
        crc ^= (uint16_t)((uint16_t)data[i] << 8);
        for (int b = 0; b < 8; b++) {
            crc = (crc & 0x8000) ? (uint16_t)((crc << 1) ^ 0x1021)
                                 : (uint16_t)(crc << 1);
        }
    }
    return crc;
}
```

4.2.2 串口流式解析与失步重同步

串口上二进制帧与控制台日志字节交织是设计预期（console REPL 与数据帧共用 USB-Serial-JTAG）。PC 端解析器不依赖任何帧间定时假设，按如下规则工作：全程搜索 0xAA 55 魔数，噪声字节计入丢弃计数；候选帧 CRC 不匹配时只前移 1 字节重扫（失步重同步），避免吞掉嵌在候选帧内部的真帧；载荷长度超过上限（1 KB）的候选头视为噪声前移 1 字节；ver 仅透传不做硬校验，CRC 是唯一准入门槛，以向前兼容版本升级。该设计经协议自测（§6.1）与实机日志交织流验证。

4.3 扰动检测算法

4.3.1 滑动窗统计与 RSSI 突变过滤

esp-radar 的数据通路分两级：逐包回调（WiFi 上下文）与检测窗回调（雷达任务上下文，台架实测约 9–10 Hz/peer）。窗口定义为两次检测窗回调之间的 CSI 包集合（约 10 包/窗）。每个源 MAC 对应一个 8 槽逐包累加器中的槽位，逐包累加单包有效子载波平均幅值 $\mathrm{amp} = \mathrm{mean}(|I + \mathrm{j}Q|)$ 的和、平方和与极值，检测窗回调到期时一次性排出，得三特征：

$$\mathrm{mean} = \frac{1}{W}\sum_{k=1}^{W}\mathrm{amp}_k, \qquad \mathrm{var} = \frac{1}{W}\sum_{k=1}^{W}\mathrm{amp}_k^2 - \mathrm{mean}^2, \qquad \mathrm{p2p} = \max_k(\mathrm{amp}_k) - \min_k(\mathrm{amp}_k)$$

其中 W 为窗内有效包数。以 var 为判决量（人扰动信道时方差增长远快于均值漂移），mean/p2p 供上位机作曲线展示与诊断。

AGC 增益跳变是本算法的主要误报源：ESP32 的 CSI 不携带 AGC 信息，射频前端增益切换表现为整窗幅值乘性跳变，无法按加性噪声滤除[7,8]。本设计在逐包回调中实施 RSSI 突变过滤：$|\Delta\text{RSSI}| > 6$ dB 的包不进滑窗，且整窗标记 dirty 直接作废（不产生特征帧、不进 FSM），计数进心跳帧的丢帧统计。累加器由 portMUX 自旋锁保护——逐包回调与检测窗回调分属不同上下文，对同一槽位的读写必须互斥。

4.3.2 滞回状态机

每条链路一个三态滞回状态机（图4.3）：IDLE 态下连续 N=10 窗 var 超阈值确认运动（≈1 s@10 Hz，抑制瞬时毛刺）；MOTION 态下连续 M=25 窗低于阈值转入 PRESENCE（2.5 s，满足“停走 3 s 内报 STILL”）；PRESENCE 表达“人刚停但可能还在”的存在语义，安静持续 10 s 后回 IDLE，期间再次连续 N 窗超阈则直接回到 MOTION。状态跳变即时发 0x10 事件帧，MOTION 持续期以 2 Hz 重发（防上位机漏收）。训练期间不做 FSM 判决，避免把标定过程当成扰动。

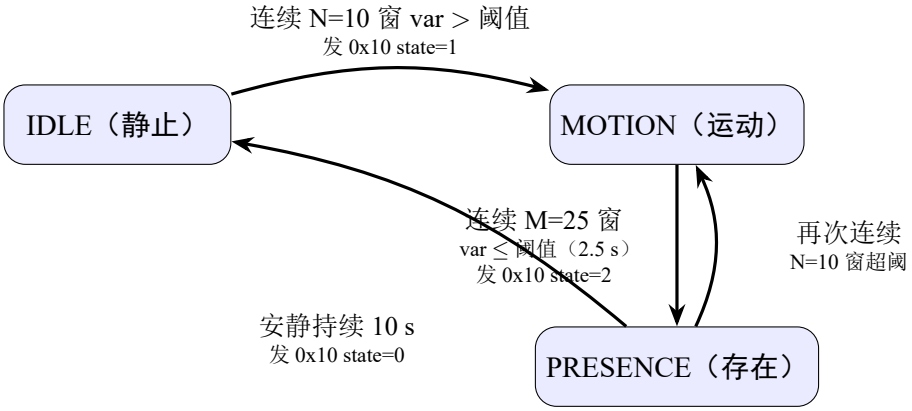


图 4.3 逐链路滞回状态机（N/M/存在保持均可经 Kconfig 配置）

4.4 现场标定与持久化

CSI 对环境高度敏感：房间几何、家具、天线位置都进入信道[6]，因此阈值不能离线写死，必须现场标定。每条链路的标定流程为：清场关门、无人静止后执行 `train <mac> 15`——固件对随后的 15 s（默认，Kconfig 可调）检测窗方差累加求均值得到噪声底，阈值按

$$T = \max(\text{噪声底} \times 300\%,\ 15)\ (\text{centi-var})$$

计算，经 0x30 标定结果帧上报上位机并同步写入 NVS；随后可用 `threshold <mac> <val>` 按验收结果 ±10–20% 微调、`threshold save` 批量持久化。链路表（源 MAC、

区域名、阈值）整体序列化为 NVS 键值（magic+ 位图 + 定长记录），上电时自动恢复并逐个重新注册 esp-radar peer。红线：标定后挪动任何一块板即链路作废，必须重标。

4.5 系统设计与微机原理知识点的对应

本系统对微机原理课程核心知识点的显性覆盖逐条对应见表4.2。

表 4.2 系统设计与微机原理知识点的对应

课程知识点	本系统中的体现	报告落点
串行通信	UART 2 Mbps（8-N-1）二进制帧上行；帧格式 AA 55/ver/-type/len/crc16；PC 端失步重同步	§4.2
校验与差错控制	CRC-16/CCITT-FALSE（poly 0x1021/init 0xFFFF）覆盖 ver. . .payload；坏帧 1 字节重扫	§4.2
中断与回调	CSI 接收回调 `wifi_radar_cb_ex`（中断上下文）、USB-Serial-JTAG 接收、TCP REPL socket 线程	§4.3、§5
定时器/计数器	`esp_timer` 1 Hz 心跳、10 Hz/peer 特征流节拍、检测窗 100 ms 粒度（10 包/窗）	§4.3
存储器组织与分区	16 MB Flash 分区表（factory@0x20000 / nvs / coredump）；NVS 键值存储链路表 + 阈值 +WiFi 凭据，断电恢复	§4.4
总线与 I/O 接口	SPI Flash、USB-Serial-JTAG、IPEX 外置天线接口、BOOT/EN 按键电路	§3.1
CPU 结构与多任务	ESP32-S3 双核 Xtensa LX7 / C3 单核 RISC-V；FreeRTOS 调度；`portMUX` 自旋锁保护 8 槽 CSI 累加器（回调/检测任务共享）	§4.3、§5
缓冲与队列	PC 端 1 MB 环形缓冲（满丢最新 + 计数）、帧队列 EVENT 优先挤占；固件侧 per-peer 累加槽	§4.6

4.6 上位机设计

上位机（`pc_dashboard/`）按“数据源抽象 + 线程模型 + 三层丢帧计数 + 卡片式 UI”组织：

- **数据源抽象**：SerialSource（串口）、UDPSource（UDP 3333）、Replay（CSV 回放）、SimSource（仿真）实现同一接口，UI 不感知具体来源；
- **线程模型**：reader 线程阻塞读串口写入 1 MB 环形缓冲（写满丢最新并计数），parser 线程取字节经 StreamParser 解帧后入帧队列；队满时 EVENT 帧优先（挤占非 EVENT 帧）；
- **三层丢帧计数**：环形缓冲丢弃字节、帧队列丢弃帧、固件心跳帧上报的固件侧丢帧计数，三层均展示于顶栏，可定位丢帧发生层次；
- **UI 布局**：顶栏（连接源/记录开关/帧率与丢帧）、左列区域状态卡片（大状态灯 +score+ 最近事件）、中央 pyqtgraph 实时曲线（每 peer 一条，事件画竖线）、底部事件表 + 命令框；标定按钮发送 `train` 并以 15 s 倒计时等待 0x30 回填阈值。

5 关键模块实现

本章按模块给出职责与关键实现，代码段均摘自工程源码（省略号表示略去的次要行），完整文件路径见附录附录 B。

5.1 固件（ESP32_CIS_V1.0）

固件模块划分见表5.1。启动序列为：`nvs_flash_init` → `cis_console_start`（REPL 先起，任何后续步骤卡住都可现场干预）→ `esp_radar_init/start`（注册两级回调）→ `cis_console_net_start`（TCP REPL，须在 esp-radar 拉起 lwip 之后调用，否则 lwip 断言重启）→ `cis_peer_load`（NVS 恢复链路表）→ `cis_wifi_start`（异步连 STA）→ `cis_detect_start` → `cis_report_start`。

表 5.1 RX 固件模块职责

模块	职责
`app_main.c`	启动序列编排，esp-radar 配置（filter_mac 引导过滤、HT20 带宽、逐包/检测窗回调注册）
`cis_detect.c`	per-peer 逐包累加器与滑窗统计、RSSI 突变过滤、滞回 FSM、训练与阈值
`cis_report.c`	5 类帧组帧发送：USB-Serial-JTAG 直写主通道 + UDP 可选；1 Hz 心跳；MOTION 期 2 Hz 重发
`cis_console.c`	REPL 命令（peer/train/threshold/report/udp/-dump/wifi_config 等）；串口 +TCP 2323 双 REPL（自实现 socket 线程）
`cis_peer.c`	链路表（8 槽）、区域名、阈值；NVS 序列化持久化；最近 CSI 源观测表
`cis_wifi.c`	STA 连网、NVS 凭据、断线重连；失败 30 s 开 SoftAP 兜底配网

5.1.1cis_detect：累加器与滑窗统计

逐包回调运行在 WiFi 上下文，必须轻量：只做单包平均幅值（有效子载波 $|I+\mathrm{j}Q|$ 的均值）计算与累加器更新。8 槽累加器按源 MAC 匹配槽位，portMUX 临界区内完成查找与累加；RSSI 突变的包不进滑窗并将整窗标记 dirty（代码5.1）。检测窗回调（约 10 Hz/peer）到期时在同一临界区一次排出窗口统计量，换算 centi 刻度后发特征帧，再进入 FSM 判决。

代码 5.1 逐包累加与 RSSI 突变过滤（cis_detect.c 节选）

```
if (slot) {
```

```
    /* RSSI 突变过滤（|ΔRSSI|>6dB 不进滑窗） */
    if (abs(rssi - slot->rssi_prev) > 6) {
        slot->dirty = true;
        slot->rssi_jump_drops++;
    } else {
        if (slot->cnt == 0) {
            slot->min_milli = slot->max_milli = mean_milli;
        } else {
            if (mean_milli < slot->min_milli) {
                slot->min_milli = mean_milli;
            }
            if (mean_milli > slot->max_milli) {
                slot->max_milli = mean_milli;
            }
        }
        slot->acc_milli    += mean_milli;             /* Sigma amp
    （毫单位） */
        slot->sq_acc_milli += (uint64_t)mean_milli * mean_milli;
        slot->cnt++;
    }
    slot->rssi_prev = (int16_t)rssi;
    slot->rssi_last = (int16_t)rssi;
}
```

检测窗回调中的滞回 FSM 判决如代码5.2：连续 N=10 窗超阈确认运动、连续 M=25 窗低阈回归、存在保持 10 s；状态跳变经 `fsm_transition` 即时发 0x10 事件帧。训练期间不进 FSM，避免把标定过程当扰动。

代码 5.2 滞回状态机判决（cis_detect.c 节选）

```
uint16_t N = CONFIG_CIS_MOTION_CONFIRM_FRAMES;
uint16_t M = CONFIG_CIS_STILL_CONFIRM_FRAMES;
bool over = d->var_c > d->threshold;

if (over) {
    d->over_cnt++;
    d->under_cnt = 0;
```

```
8  } else {
9      d->under_cnt++;
10     d->over_cnt = 0;
11     if (d->state != CIS_DET_IDLE) {
12         d->quiet_ms += (ts > d->last_ts_ms) ? (ts - d->last_ts_ms)
       : 0;
13     }
14 }
15
16 switch (d->state) {
17 case CIS_DET_IDLE:
18     if (over && d->over_cnt >= N) {
19         fsm_transition(d, CIS_DET_MOTION, ts);
20     }
21     break;
22 case CIS_DET_MOTION:
23     if (d->under_cnt >= M) {
24         fsm_transition(d, CIS_DET_PRESENCE, ts);
25     }
26     break;
27 case CIS_DET_PRESENCE:
28     if (over && d->over_cnt >= N) {
29         fsm_transition(d, CIS_DET_MOTION, ts);
30     } else if (d->quiet_ms >= (uint32_t)CONFIG_CIS_PRESENCE_HOLD_S
       * 1000u) {
31         fsm_transition(d, CIS_DET_IDLE, ts);
32     }
33     break;
34 default:
35     d->state = CIS_DET_IDLE;
36     break;
37 }
```

5.1.2cis_report：帧发送与心跳

检测上下文产生的特征/事件/标定结果先入消息队列，由独立上报任务组帧发送，避免在雷达回调里做串口写与 NVS 写。主通道直写 USB-Serial-JTAG 硬件接口，绕过

VFS 层，避免控制台 CRLF 翻译破坏二进制帧；UDP 为可选副通道（默认关，端口 3333），同帧格式。心跳由 1 Hz 节拍产生，携带运行时长、peer 数、空闲堆内存与固件侧丢帧计数，作为 PC 端链路存活判断与丢帧对账的依据；MOTION 事件在持续期以 2 Hz 重发，防止上位机漏收单个事件帧。

5.1.3cis_console：双 REPL

REPL 双开：主 REPL 走 USB-Serial-JTAG（COM3 双向）；无线 REPL 为 cis_console 自实现的 TCP socket 线程（IDF v5.5.6 的 esp_console 没有 `esp_console_new_repl_net` API），并把日志 vprintf 镜像到 TCP 连接，便于无线标定与观测。命令经 esp_console 注册，覆盖链路登记（peer add/del/list/scan）、标定（train、threshold[save]）、上报控制（report、udp）与诊断（dump、wifi_status/wifi_scan、system restart）。

5.1.4cis_peer：链路表与 NVS 持久化

链路表容量 8 槽，每槽记录源 MAC、区域名与阈值。NVS 序列化格式为 magic（“CIS1”）+ 槽位使用位图 + 定长记录（代码5.3），一次读写一个二进制块。上电时 `cis_peer_load` 反序列化并校验 magic 后逐槽重建 esp-radar peer 与检测上下文，实现断电恢复。

代码 5.3 链路表 NVS 序列化（cis_peer.c 节选）

```
/* NVS 序列化：magic(4) + used 位图(1) + 每槽 [mac6|name16|thr2] */
static void table_serialize(uint8_t *blob)
{
    memset(blob, 0, BLOB_SIZE);
    blob[0] = (uint8_t)(NVS_MAGIC & 0xFF);
    blob[1] = (uint8_t)((NVS_MAGIC >> 8) & 0xFF);
    blob[2] = (uint8_t)((NVS_MAGIC >> 16) & 0xFF);
    blob[3] = (uint8_t)((NVS_MAGIC >> 24) & 0xFF);
    uint8_t used_bits = 0;
    for (int i = 0; i < CIS_PEER_MAX; i++) {
        uint8_t *rec = blob + 5 + i * REC_SIZE;
        if (s_table[i].used) {
            used_bits |= (uint8_t)(1U << i);
            memcpy(rec, s_table[i].mac, 6);
            memcpy(rec + 6, s_table[i].name, CIS_PEER_NAME_LEN);
            rec[6 + CIS_PEER_NAME_LEN] = (uint8_t)(s_table[i].
    threshold & 0xFF);
```

```
            rec[6 + CIS_PEER_NAME_LEN + 1] = (uint8_t)(s_table[i].
    threshold >> 8);
        }
    }
    blob[4] = used_bits;
}
```

5.1.5cis_wifi：连网与兜底

事件回调处理 STA 启动/断线/IP 获取三类事件：断线即重连；连网任务等待事件组连接标志，30 s（SOFTAP_FALLBACK_MS）未连上则启动 SoftAP（CIS_RX-ae10）兜底配网。WiFi 凭据存 NVS，重启后自动重连。

5.2 上位机（pc_dashboard）

5.2.1protocol：协议编解码与流式解析

protocol.py 为纯标准库实现，dataclass 载荷字段序与线上字节序一致（struct.Struct 显式小端）；表驱动 CRC 与固件逐位实现交叉验证（自测用）。流式解析器的准入与重同步核心如代码5.4：CRC 不匹配只前移 1 字节，保留嵌在其后的真帧。

代码 5.4 流式解析：CRC 准入与失步重同步（protocol.py 节选）

```
body = bytes(buf[2:HEADER_LEN + plen])
(crc_got,) = struct.unpack_from("<H", buf, HEADER_LEN + plen)
if crc16_ccitt(body) != crc_got:
    # CRC 不符：只前移 1 字节，保留嵌在其后的真帧
    self.bad_crc += 1
    self.resyncs += 1
    self.bytes_dropped += 1
    del buf[:1]
    return None, True
payload = bytes(buf[HEADER_LEN:HEADER_LEN + plen])
del buf[:total]
self.frames_ok += 1
return Frame(ver=ver, type=ftype, payload=payload), True
```

5.2.2serial_source：双线程与环形缓冲

reader 线程阻塞读串口写入 1 MB 环形缓冲，写满时丢弃装不下的最新部分并计数（保旧数据完整，parser 消费连续，不会读到半截被覆盖的数据）；parser 线程取出字节喂 StreamParser，解出的帧入帧队列。帧队列满时的 EVENT 优先挤占逻辑如代码5.5——事件帧是监控系统的核心输出，宁可丢特征帧不丢事件帧。

代码 5.5 帧队列 EVENT 优先挤占（serial_source.py 节选）

```
def _enqueue(self, frame: protocol.Frame):
    """队满时：EVENT 优先——挤掉队首非 EVENT 帧；其余帧直接丢弃计数。"""
    while True:
        try:
            self.frame_queue.put_nowait(frame)
            return
        except queue.Full:
            if frame.type == protocol.TYPE_EVENT:
                try:
                    self.frame_queue.get_nowait()
                except queue.Empty:
                    continue
                self._stats["queue_dropped"] += 1
            else:
                self._stats["queue_dropped"] += 1
                return
```

5.2.3store：CSV 会话与回放缓冲

CSV 会话文件为 `data/YYYYMMDD_HHMM/cis_log.csv`，10 列表头（ts_iso, mac, name, type, state, score, mean, var, p2p, rssi）为并集 schema——各帧类型只填自己有的列，其余留空，表头一致性即“CSV 列齐全”判据（`Store.header_matches`）。所有帧归一化为 Record 后同时写入 CSV 与内存回放缓冲（默认 3 万条，约两 peer 特征流 25 分钟），跨文件回放由 replay.py 按时间戳倍速重现。

5.2.4ui_main：监控界面

ui_main 基于定时轮询循环消费帧队列，按帧类型分发（代码5.6）：EVENT 更新事件表与事件竖线、FEATURES 追加每 peer 曲线、HEARTBEAT 更新顶栏存活与丢帧读

数、CALIB 回填标定结果；任何带 MAC 的记录同时刷新对应区域卡片并存入 CSV。区域卡片按源 MAC 动态创建，缺省名以 MAC 末 4 位生成。

代码 5.6 帧分发（ui_main.py 节选）

```
def _dispatch(self, rec: Record):
    if rec.mac:
        self._last_seen[rec.mac] = rec.ts
        card = self._get_card(rec.mac)
        card.update_record(rec)
    if self.store.is_recording:
        self.store.record(rec)

    kind = rec.kind
    if kind == "HEARTBEAT":
        self._hb = rec.raw
    elif kind == "EVENT":
        self._add_event_line(rec)      # 曲线事件竖线
        self._add_event_row(rec)       # 事件表
    elif kind == "FEATURES":
        s = self._series.setdefault(rec.mac, {...})
        s["t"].append(rec.ts)
        s["var"].append(rec.var or 0)
    elif kind == "CALIB":
        self._add_event_row(rec)
        self.log("CALIB %s %s" % (rec.mac, rec.text))
        if self._calib and self._calib["mac"] == rec.mac:
            self._finish_calib(rec)
    elif kind == "LOG":
        self.log("RX: %s" % rec.text)
```

6 测试与验证

本章汇总台架阶段已完成并留有日志证据的实验；空间部署后的现场验收（第7章）尚未执行，不计入本章。全部数据的证据文件位于工程 `docs/` 目录，标注于各节。

6.1 协议自测

`pc_dashboard/protocol_selftest.py`（纯标准库，可作确定性门控）覆盖五类用例：编解码往返一致性、坏帧拒绝、失步重同步、控制台日志字节交织容忍、CRC 变体交叉验证（表驱动 vs 逐位实现、标准校验值 0x29B1）。实测 **20/20 用例全部通过**（证据：`pc_dashboard/protocol_selftest.py`，2026-10-28 运行记录）。

6.2 数据流与帧质量

M1 数据流打通。COM3@2 Mbps 录制 60 s：546 帧全部来自 TX1（MAC 归因 100% 正确），无断流、零 >300 ms 间隙；包到达间隔中位数 110 ms、最大 132 ms（对应检测窗约 10 包/窗的粒度）；幅值 mean = 37.93、std = 0.099、p2p = 0.64（raw 刻度），RSSI 恒为 −6 dBm、AGC 增益恒 0——近距静态链路高度稳定（证据：`m1_sample.log`）。此前的基线记录 `m1_sample_prefixfix_baseline.log` 暴露了 v1 打印的整数均值截断伪影（真值 36.5–37.5 恒打出 37，std = 0 假象），v2 改毫单位打印并增加窗内 p2p 与 agc_gain 诊断字段后消除。

M2 帧质量。60 s 串口流解析统计：心跳 59 帧（间隔恒定 1000 ms）、事件 116 帧、特征 533 帧（约 9 Hz/peer，与 esp-radar 窗口率一致），**0 坏 CRC**（证据：`m2_verify_run7.log`）。双链路复核抽样：60 s 特征帧 1082 帧（两 MAC 各 541 帧，约 9 Hz/peer）、心跳 59 帧、0 坏 CRC，特征流速率与设计一致（证据：`a6_m2_0350.log`）。

无线扰动恢复。执行 `wifi_scan` 使 RX 跳扫离开信道，特征流停止 2.6 s；扫描回到工作信道后特征流立即恢复（证据：`m2_verify_run11_scan.log`）。该实验同时验证了信道一致性是 CSI 链路收发的前提——M2 调试期曾因 RX 固定信道 6 而 AP 实际在信道 4 而静默无数据，最终以“STA 跟随 AP 信道”解决。

上电恢复。RX 重启后链路表从 NVS 恢复、TCP console 进入监听、随后获取 IP（证据：`m2_verify_run10_boot.log`）。

6.3 双链路联调与 NVS 恢复

双链路标定。TX1/TX2 注册为 zone1/zone2 后分别清场标定（`train 15`）：TX1 阈值 681、噪声底 227；TX2 阈值 20367、噪声底 6789（centi-var）——两链路噪声底差异近 30 倍，直观证明逐链路独立标定的必要性；`threshold save` 持久化成功（证据：

m4_register_train_run1.log）。

串扰隔离台架证据。复位 TX2 制造 5.08 s 静默窗，期间 TX1 特征流零间隙，全部事件帧的 MAC 归因正确——证明“按源 MAC 分发 + per-peer 上下文”的链路隔离设计有效（证据：m4_register_train_run1.log，解析脚本 scripts/m4_analyze.py）。需要说明：这是近距台架下的功能级证据，空间部署后的“分别遮挡”串扰验收仍按第7章用例 □ 执行。

NVS 重启恢复。软重启（rst:0xc）后，双 peer 携带阈值在 1.26 s 内恢复（peer list 回读 thr=681/20367 与断电前一致），4.9 s 时获取 IP（证据：m4_restart_nvs_run4.log），远优于“断电恢复 ≤30 s”的指标承诺。

长程帧统计。双链路会话累计 1860 帧 0x11 特征帧 + 293 帧 0x10 事件帧，**0 坏 CRC**（证据：m4_register_train_run1.log）。

6.4 上位机冒烟

pc_dashboard/ui_smoke.py 以 SimSource 仿真数据源驱动完整 UI：实测 **17/17 检查项通过**，运行 60 s 后自动退出、退出码 0；生成的 CSV 首行与 10 列表头定义精确一致（Store.header_matches）。回放缓冲、事件表、曲线数据结构与卡片状态机在冒烟中同步覆盖（证据：pc_dashboard/ui_smoke.py，2026-10-28 运行记录）。上位机实机串口运行截图见第7章占位。

6.5 台架验证小结

综合上述实验：协议两端实现一致（20/20）；数据链路在 2 Mbps 串口上长期零 CRC 错误；特征流速率、心跳节拍与设计一致；事件 MAC 归因与链路隔离有效；NVS 断电恢复链路完整。台架功能验证全部通过，指标级验证（检出率、误报率、丢帧率、恢复时间在真实空间下的取值）待现场验收。

7　现场部署与验收

本章为空间部署后的现场验收安排。除部署方法与判据（来自标定 SOP 与验收记录表，属设计内容）外，**全部结果待现场执行后填写**，当前一律以红色占位标注，不预设任何数值。

7.1 部署点位

部署遵循标定 SOP：每个监测点 1 块 TX，TX 与 RX 分居活动区两侧、链路中点覆盖活动区；天线竖直同极化；离地 ≥1 m、离墙与金属大反射体 ≥0.5 m；胶带标记位置并拍照存档。路由器 2.4 GHz 信道手动固定、20 MHz 带宽、关闭自动信道与频段合一。

【实测待填：场地信息——房间/楼层/尺寸/路由器实际信道】

【实测待填：部署点位图与点位照片】

图 7.1 部署点位图与照片（待现场拍摄替换）

7.2 现场标定结果

按 SOP 每链路 ≤30 s 完成 `peer add` → 清场 `train 15` → `threshold save`，结果填入表7.1并逐链路复验（静置 30 s 无事件、走动 2 s 内报 MOTION、停走 3 s 内报 STILL）。

表 7.1 现场标定结果表（待现场填写）

链路	CSI 源 MAC	区域名	噪声底（c）	阈值（c）	threshold save
1	【实测待填】	【实测待填】	【实测待填】	【实测待填】	【实测待填】
2	【实测待填】	【实测待填】	【实测待填】	【实测待填】	【实测待填】
3	【实测待填】	【实测待填】	【实测待填】	【实测待填】	【实测待填】

7.3 七条验收用例结果

验收按 `docs/□□□□□.md` 逐条执行，判定口径为通过/降级/不通过（隔墙、微动等场景允许注明降级与灵敏度极限，不许调参凑过）。七条用例及判据见表7.2，结果待现场

填写。

表 7.2 七条验收用例与结果（结果待现场填写）

号	用例	判据	结果
1	静置 30 min 误报	任一 10 min 窗误报 ≤2 次	【实测待填】
2	0.5 m/s 横穿 10 次	≥9 次 2 s 内报 MOTION 且 MAC 正确	【实测待填】
3	1–2 m 挥手	3 s 内检出（检不出记灵敏度极限）	【实测待填】
4	隔墙走动	重标定后走动可检（存在检测可降级）	【实测待填】
5	双区并发	事件 MAC 归因全程正确、无串扰	【实测待填】
6	10 min 丢帧率	<1%（上位机三层计数与心跳对账）	【实测待填】
7	断电重启	≤30 s 恢复且链路表/阈值一致	【实测待填】

7.4 30 分钟长稳与上位机实机运行

长稳与用例 1 同跑：上位机连续记录 30 min，观察无崩溃/卡死、CSV 持续写入、顶栏帧率与丢帧读数正常。

【实测待填：上位机实机运行截图——区域卡片/实时曲线/事件表】

【实测待填：30 min 长稳曲线截图（var 曲线 + 事件竖线 + 丢帧读数）】

图 7.2 上位机实机运行与长稳截图（待现场截取替换）

【实测待填：演示照片/视频截图】

8 总结与展望

8.1 工作总结

本文完成了基于 ESP32 CSI 扰动特征的多监测点监控系统的设计与实现：以 esp-radar 组件的多 peer 能力按源 MAC 分发 CSI，逐链路实现滑动窗统计（mean/var/p2p）+ RSSI 突变过滤 + 三态滞回状态机的扰动检测；设计了带 CRC-16 校验、centi 定点刻度与失步重同步的 2 Mbps UART 二进制上报协议；实现了现场标定（train 15 s、阈值下限钳制）与 NVS 断电持久化；开发了 PySide6+pyqtgraph 桌面上位机（区域卡片、实时曲线、事件日志、CSV 记录/回放、三层丢帧计数）。

台架阶段完成了五组实验并全部通过：协议自测 20/20；单链路 60 s 采集 546 帧无断流、包间隔中位数 110 ms；60 s 心跳 59 帧、特征 533 帧、零坏 CRC；双链路独立标定（阈值 681/20367 centi-var）与 TX 复位静默窗下的 MAC 归因正确性验证；断电重启 1.26 s 链路表恢复、4.9 s 联网；上位机冒烟 17/17。系统在微机原理课程知识点的覆盖上显性对应了串行通信、校验与差错控制、中断与回调、定时器、存储器组织、I/O 接口、多任务与并发保护、缓冲与队列八个方向（表4.2）。

8.2 局限与已知问题

如下局限须如实说明，其中多条源于 CSI 感知的物理特性而非实现缺陷：

1. **台架布局的功能验证局限**：联调为近距台架（板间距 <0.5 m，RSSI 饱和），仅验证了功能与数据流正确性；检出率/误报率等正式指标须空间部署后重标定，且标定后挪动即作废。
2. **不可区分人数与身份**：系统检测的是“每条链路的扰动”，无法区分人数或身份——这是商品级 ESP32 CSI 的公认物理局限[5]，多节点系统的预期按“每链路独立”管理。
3. **环境敏感**：CSI 对环境高度敏感，换场地必须重新 train；信道跟随 AP（当前实测信道 4），跨场地泛化是 Wi-Fi 感知的公认难题[6]。
4. **两次未归因重启**：联调期间 RX 出现两次来源未明的重启，疑 USB 供电跌落（未证实），已列入部署排查项，对策为 TX/RX 独立供电与短粗 USB 线。
5. **TX3 待恢复**：经典 #2 芯片过热进不了下载模式，待冷却恢复后重烧固件；三链路完整联调待其恢复后进行。

8.3 展望

算法层面，可对标 ESPectre 框架升级为“标定期筛选高方差子载波 + 仅对 top-K 子载波做滑动方差”，比全子载波平均有更高信噪比[4]（验收后增强项）。工程层面，完

成三链路（含 TX3 恢复）完整联调与直收模式演练，使系统具备无路由场地的答辩能力；并观察 RSSI 突变过滤的实机触发情况（台架未触发，丢帧计数无增量）。应用层面，802.11bf 已正式批准[3]，本系统可作为该标准感知能力的最小可用子集展示平台，向多房间联动与呼吸/存在细粒度检测扩展。

9 分工

小组成员与分工如下（待填写）：

- **【实测待填：成员 1 姓名——分工】**
- **【实测待填：成员 2 姓名——分工】**
- **【实测待填：成员 3 姓名——分工】**

分工建议按系统构成划分：固件（检测/上报/控制台）、上位机（协议/界面/存储）、硬件与现场（板卡管理、烧录备份、部署标定、验收执行）、报告撰写与图表，供小组按实际参与情况填写。

附录 A 协议帧格式总表

帧外层格式统一为：AA 55 | ver(1B = 0x01) | type(1B) | len(2B LE) | payload | crc16(2B LE)，CRC-16/CCITT-FALSE 覆盖 ver...payload。各帧类型 payload 逐字节定义见表a-1（多字节字段一律小端，u16 饱和刻度见 §4.2）。

表 a-1 各帧类型 payload 逐字节定义（偏移自 payload 首字节）

类型	偏移	字段	长度	说明
0x01 心跳	0	uptime_ms	4	上电运行毫秒数
	4	peer_count	1	已注册链路数
	5	free_heap	4	空闲堆内存（B）
	9	lost_frames	2	固件侧丢帧累计
0x10 事件	0	mac	6	CSI 源 MAC
	6	name_id	1	区域名编号（链路表槽位）
	7	state	1	0 静止 / 1 运动 / 2 存在
	8	score	2	触发时窗方差（centi-var）
	10	rssi	1	有符号 dBm
	11	ts_ms	4	触发时刻（ms）
0x11 特征	0	mac	6	CSI 源 MAC
	6	ts_ms	4	窗口时刻（ms）
	10	mean	2	窗均值（centi-amp）
	12	var	2	窗方差（centi-var）
	14	p2p	2	窗峰峰值（centi-amp）
	16	rssi	1	有符号 dBm
0x20 日志	0	text	len	ASCII 文本回显
0x30 标定	0	mac	6	CSI 源 MAC
	6	train_s	2	标定时长（s）
	8	threshold	2	阈值（centi-var，已存 NVS）
	10	noise_floor	2	噪声底（centi-var）

附录 B 工程目录与证据文件索引

B.1 工程目录索引

工程根目录 `F:\ESP\projects\ESP32-CIS\`，与报告对应的源码位置：

表 b-1 工程目录与报告章节对应

目录	内容	对应章节
`ESP32_CIS_V1.0/main/`	RX 固件（cis_protocol.h、cis_detect / cis_report / cis_console / cis_peer / cis_wifi、app_main.c、Kconfig.projbuild）	§5、§4.2、§4.3
`csi_tx_v1/`、`csi_tx_c3_v1/`	TX 固件（csi_send 示例改配置，源 MAC 各异）	§3.1
`pc_dashboard/`	protocol.py、serial_source.py、store.py、udp_source.py、replay.py、ui_main.py、ui_smoke.py、minimal_monitor.py、protocol_selftest.py	§4.6、§5
`scripts/`	flash_rx.bat、flash_tx.bat（MAC 门禁 + 源 MAC 防呆）、backup_flash.sh、m1/m2/m4 分析脚本	§3.1、§6
`docs/`	实施计划、调研报告、标定 SOP、验收记录表、全部日志证据	全文

B.2 证据文件索引

第6章全部实验的证据文件（位于 `docs/`）：

- `m1_sample.log`、`m1_sample_prefixfix_baseline.log` ——M1 数据流（546 帧/60 s、间隔统计、修复前后基线）；
- `m2_verify_run7.log` ——M2 帧质量（心跳 59/事件 116/特征 533，0 坏 CRC）；
- `m2_verify_run11_scan.log` ——wifi_scan 跳扫离信道 2.6 s 与恢复；
- `m2_verify_run10_boot.log` ——重启后 NVS 恢复、TCP console 监听、got IP；
- `m4_register_train_run1.log` ——双链路标定（681/227、20367/6789）、TX2 复位静默窗串扰隔离、1860+293 帧零坏 CRC；
- `m4_restart_nvs_run4.log` ——软重启后双 peer 带阈值 1.26 s 恢复、4.9 s got IP；
- `a6_m2_0350.log` ——双链路复核抽样（特征 1082 帧、两 MAC 各 541）；
- `pc_dashboard/protocol_selftest.py`、`pc_dashboard/ui_smoke.py` ——协议自测（20/20）与上位机冒烟（17/17）；
- `scripts/m4_analyze.py` ——M4 静默窗/归因分析脚本。

B.3 【实测待填】占位清单

全文红色占位（搜索“【实测待填”可定位）与任务书占位清单对应如下，交付现场数据后逐项替换：

表 b-2 占位清单

号	占位项	位置
1	封面题目页全部字段（学号/姓名/小组成员/学院/专业/指导教师/日期）	封面
2	实物照片（RX/TX 板卡、天线、整机）	图3.2
3	场地信息（房间/尺寸/信道）与部署点位图、点位照片	§7、图7.1
4	三链路现场标定值表（噪声底/阈值/保存确认）	表7.1
5	七条验收用例全部数据	表7.2
6	30 min 长稳曲线截图	图7.2
7	上位机实机运行截图（卡片/曲线/事件表）	图7.2
8	演示照片/视频截图	§7
9	小组成员与分工	第8章之后

参考文献

[1] Espressif Systems. esp-csi: ESP32 CSI 基于 Wi-Fi 信道状态信息的感知工具集与 esp-radar 组件 [CP/OL]. https://github.com/espressif/esp-csi, 2025.

[2] Hernandez S M, Bulut E. WiFi Sensing on the Edge: Signal Processing Techniques and Promise for Real-Time RNG[J]. IEEE Communications Surveys & Tutorials, 2022, 25(1): 330–362.

[3] Du R, Santoro D, Seifert M, et al. An Overview on IEEE 802.11bf: WLAN Sensing[EB/OL]. arXiv:2207.04859, 2022.（IEEE 802.11bf 标准已于 2025 年 5 月正式批准）

[4] Pace F, et al. ESPectre: ESP32 CSI Motion Detection Framework[CP/OL]. https://espectre.dev, 2025.

[5] Multi-Person Gait Separation with Commodity WiFi Sensors[EB/OL]. arXiv:2601.02177, 2026.

[6] A Survey on Wi-Fi Sensing Generalizability[EB/OL]. arXiv:2503.08008, 2025.

[7] CSI Simulation: Why Additive Noise Fails and How to Fix It[EB/OL]. arXiv:2607.01882, 2026.

[8] Espressif. esp-csi Issue #185: AGC 增益跳变对 CSI 幅值的影响与对策 [EB/OL]. https://github.com/espressif/esp-csi/issues/185, 2024.

[9] Espressif Systems. ESP-IDF Programming Guide (v5.5.6): ESP-WIFI CSI / NVS / esp_console / FreeRTOS[EB/OL]. https://docs.espressif.com/projects/esp-idf/en/v5.5.6/esp32s3/, 2025.

[10] Espressif Systems. ESP32-S3 / ESP32-C3 技术参考手册 [Z]. 2024.